

The polymorphic menagerie

Class hierarchies, relationally caged

Andrei Lepikhov

pgEdge

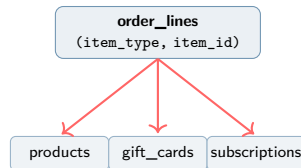
2026



The polymorphic menagerie

Class hierarchies, relationally caged

A scene you've seen before. The front page of any catalog-driven application: a top- N ranking of items shown together with their per-category details — an air fryer, a vitamin supplement, protein brownies — each row resolving to its own subtype table.



No FK spans multiple targets
⇒ encoding workaround

The same query shape elsewhere:

- **ORMs.** Rails *polymorphic associations*, Django *polymorphic models*, Hibernate `@Inheritance(JOINED)`
- **CRM.** Salesforce WhoId / WhatId polymorphic lookups
- **1C platform.** virtually every catalog-shaped configuration in production

Why it blows up: the per-row mismatch

The discriminator picks one target per row.
The planner picks one probe order for *all* rows.

	products	gift_cards	subscriptions	
item_type='product'	✓	×	×	useful: $1/N$
item_type='gift_card'	×	✓	×	
item_type='subscription'	×	×	✓	
wasted: $(N-1)/N$ per row				

The discriminator's mutual-exclusion structure is invisible to the planner.
Every row pays for all N probes — the diagonal is the only useful work;
the off-diagonal is forced waste.

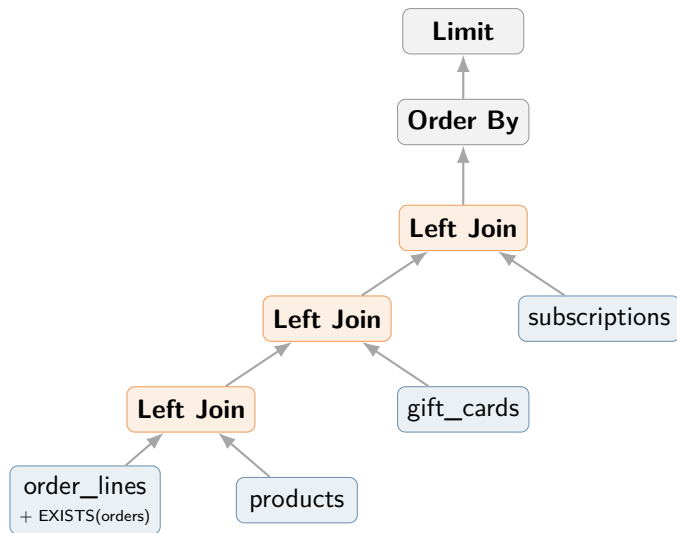
The Query

```
SELECT
    ol.id,
    COALESCE(p.name, g.name, s.name) AS item_name
FROM order_lines ol
LEFT JOIN products p
    ON ol.item_type = 'product' AND ol.item_id = p.id
LEFT JOIN gift_cards g
    ON ol.item_type = 'gift_card' AND ol.item_id = g.id
LEFT JOIN subscriptions s
    ON ol.item_type = 'subscription' AND ol.item_id = s.id
WHERE EXISTS (
    SELECT 1 FROM orders o
    WHERE o.id = ol.order_id AND o.placed_at >= --2024\01\01)
ORDER BY ol.popularity
LIMIT 100
```

Typical optimiser choices

- NestLoop LEFT JOIN type because of PK index scan on the inner side

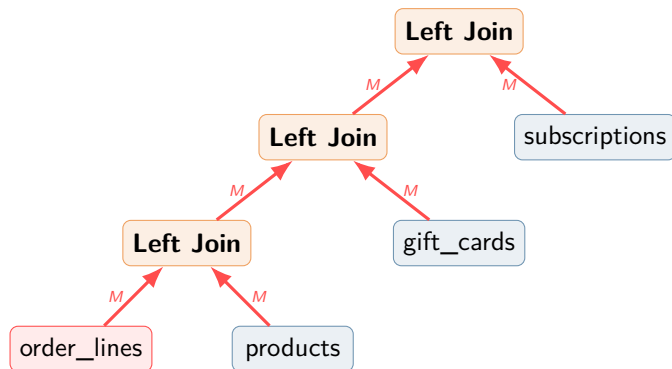
Query Plan Tree



Main Execution Cost Amplifiers

- Base table (`order_lines`) growth
- Ordering/grouping column of the base table not covered by an index
- Growth in the number of LEFT JOINs (target types)
- Unstable placement of the EXISTS filter

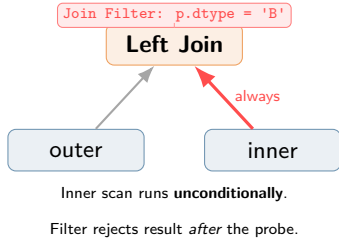
Amplifier: Base Table Growth



Every edge carries *M* rows. Doubling *M* doubles number of probes on the inner side.

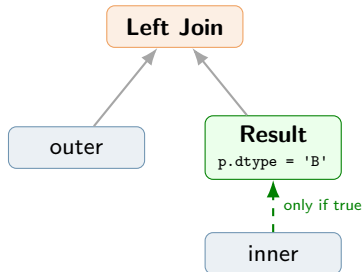
Concept: Result Filter (from pgsql-hackers)

Current: probe, then filter



Thread on pgsql-hackers

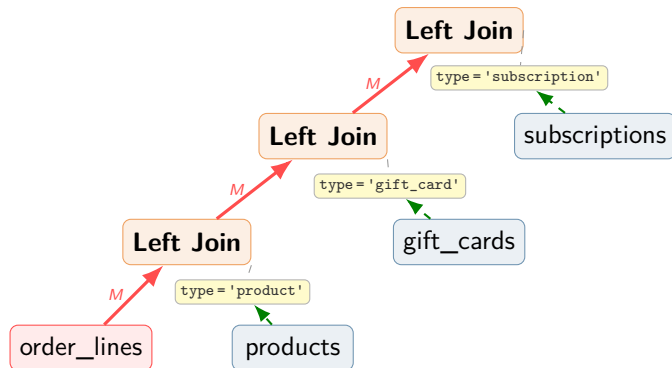
Proposed: Result filter atop RHS



Result node evaluates LHS-only clause as a **one-time** filter.

If false → returns $\emptyset$; inner scan *never* launched.

One-Sided Clause Guard

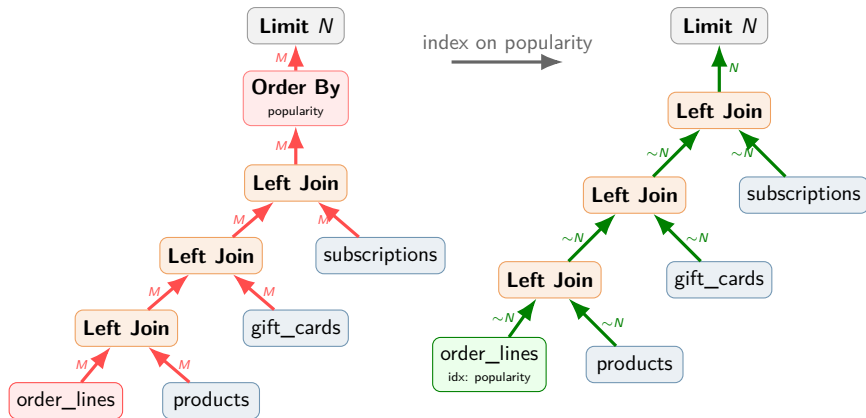


Guard checks discriminator *before* probing inner side. Each row probes only the matching table: $M_1 + M_2 + M_3 = M$.

Benefits of the Result Filter Approach

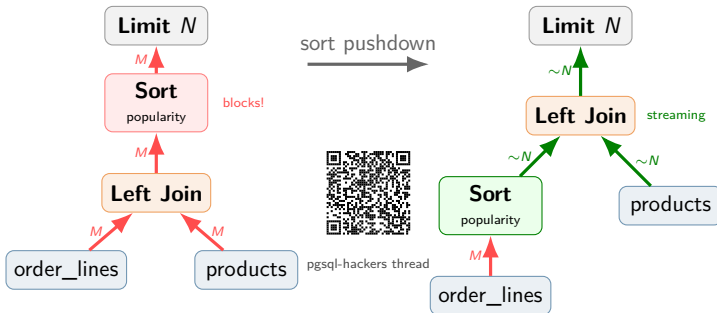
- **Fewer page accesses.** Each base row probes only the *matching* inner table instead of all N . Total index/heap page reads drop from $M \times N$ to M .
- **Safety net for missing indexes.** When no index covers the inner side, the executor would fall back to a sequential scan *per outer row*. The guard lets it skip the scan entirely for non-matching rows.

ORDER BY + LIMIT: Effect of Proper Index

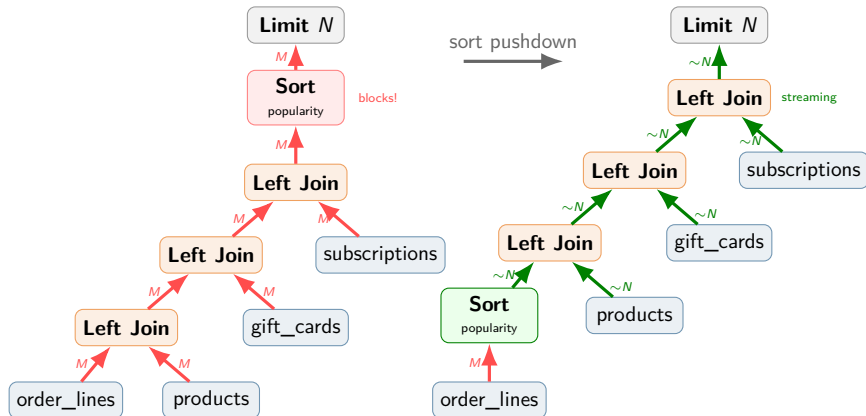


$N = LIMIT$. Index-ordered scan eliminates the blocking `Order By`. `Limit` stops the entire `LEFT JOIN` chain after N rows: $M \times 3 \rightarrow \sim N \times 3$ probes.

What if no proper index exists?



Sort Pushdown: Presorted Outer Path



Push Sort below the joins: base rows sorted once ($M \log M$), but only $\sim N$ enter the join chain. Probes: $M \times 3 \rightarrow \sim N \times 3$.